



The Identical Parallel Machine Scheduling Problem with Setups and Additional Resources

Ângelo Soares¹ , Ana Rita Ferreira¹ , and Manuel Pereira Lopes^{1,2}  

¹ ISEP, Polytechnic of Porto, R. Dr. António Bernardino de Almeida, 4249-015 Porto, Portugal
mpl@isep.ipp.pt

² INESC TEC, R. Dr. Roberto Frias 400, 4200-465 Porto, Portugal

Abstract. This paper studies a real world dedicated parallel machine scheduling problem with sequence dependent setups, different machine release dates and additional resources (PMSR). To solve this problem, two previously proposed models have been adapted and a novel objective function, the minimisation of the sum of the machine completion times, is proposed to reflect the real conditions of the manufacturing environment that motivates this work. One model follows the strip-packing approach and the other is time-indexed. The solutions obtained show that the new objective function provides a compact production schedule that allows the simultaneous minimisation of machine idle times and setup times. In conclusion, this study provides valuable insights into the effectiveness of different models for solving PMSR problems in real-world contexts and gives directions for future research in this area using complementary approaches such as matheuristics.

Keywords: Parallel machines · Scheduling · Setups · Additional Resources

1 Introduction

The study of Parallel Machine scheduling problems (PM) has mostly relied on machines as the primary resource, neglecting the fact that real manufacturing settings involve other critical resources such as automated guided vehicles, machine operators, tools, pallets, dies, and industrial robots [1–3]. The PM with sequence dependent setup times (PMS) where additional processing and setup resources are required (PMSR) is more complex. It involves sequencing and timing decisions and balancing multiple performance metrics. This study focuses on the injection moulding department of an automotive company that produces plastic injection moulded parts for final assembly plants with a maximum response time of 48 h. To solve this real problem, two existing models are adapted to the PMSR involving identical PM with different machine release dates, and a new objective function that minimises the sum of the completion times for each machine is proposed. This objective function provides a compact production schedule that allows the simultaneous minimisation of machine idle times and setup times for the specific case of dedicated PM with sequence dependent setup times. To the authors' knowledge, no existing research has explored the minimization of the sum of completion times for each machine as an objective function. This study provides valuable insights into the

effectiveness of different models for solving this type of PMSR problem in real-world contexts.

2 Literature Review

In recent years, several papers have addressed the more general case of PMSP with setups and resources. Akyol Ozer et al. [4] studied the identical PMSP with sequence-dependent setup times, machine eligibility restrictions and multiple copies of shared resources. In this paper, the objective function was the minimization of the total weighted completion time. For small instances (8 jobs, 3 machines and 5 resources), a mixed integer linear programming model (MILP) obtained optimal solutions in almost all tests. For larger problems, a matheuristic consisting of a MILP and a Genetic Algorithm (GA) has been implemented, which provides solutions for up to 100 jobs, 8 machines and 50 resources. Bektur et al. [5] presented a MILP and two metaheuristics, Tabu Search (TS) and Simulated Annealing (SA), for the case of a team as a setup resource and the objective of minimizing the total weighted tardiness. MILP only gives optimal solutions for 2 machines and 10 jobs with a time limit of 7200 s. Regarding the metaheuristics, for up to 10 machines and 100 jobs, TS outperformed SA with an average relative deviation of 5.58%. Pinheiro et al. [6] propose a solution approach to the unrelated PMSP with family setups and resource constraints, to minimize the total tardiness of the jobs. The authors develop a MILP and two metaheuristics, SA and Iterated Greedy (IG). The MILP showed good performance for instances with up to 20 jobs and 6 machines, with a computational time limit of 3600 s. To address larger problems, a constructive heuristic was used to provide an initial solution, which was then improved by the metaheuristics. For instances with up to 50 jobs, the IG algorithm outperformed the SA algorithm, exhibiting lower relative deviation values. In fact, IG and SA algorithms have average relative deviations of 8.84% and 39.87%, respectively. The computational time required for the IG algorithm to provide a solution was also reported to be lower than the MILP model. Fanjul-Peyro [7] addressed the Unrelated PM with Setups and Resources (UPMSR), with three types of resources: specific for processing, specific for setup and shared. The authors presented two MILP models and a three-phase exact algorithm (TPhA) to improve the model's performance. Yepes-Borrero et al. [8] implemented a GRASP algorithm with the objective of minimizing makespan and in [9] have also proposed a multi-objective approach to minimise both the makespan and the number of setup resources, building on the previous work [7, 8]. An adaptation of the Restarted Iterated Pareto Greedy (RIPG) algorithm is proposed that gives solutions for up to 250 jobs and 30 machines. Zhang et al. [10] tackled the unrelated PMSP, with limited worker resources and learning effect. They proposed a Combinatorial Evolutionary Algorithm to minimize makespan and energy consumption. The algorithm can handle up to 400 jobs, 22 machines, and 10 resources. Lopez-Esteve, et al. [11] also developed GRASP algorithms to solve the unrelated PMSP with additional resources during processing and setups which gives solutions with a gap of 1.80% in relation to the Fanjul-Peyro [7]. Yunusoglu, et al. [12] proposed an exact model based on constraint programming (CP) to solve the multi-resource-constrained unrelated PMSP with sequence-dependent setup times. The objective was to minimize the makespan and release dates were incorporated

into the problem as operational constraints to reflect real-world manufacturing environments. The computational results indicate that the proposed CP model outperforms the best solutions of previous works with an average gap of 15.52%. Finally, Afzalirad, et al. [13] inspired on the block erection scheduling problem in a shipyard, studied the unrelated PMSP with resource constraints, sequence-dependent setup times, different release dates, machine eligibility and precedence constraints, to minimize the makespan. The results indicate that in the case of small-scale problems, both methods are efficient and effective, whereas in large-scale problems, the AIS approach statistically outperforms the GA approach.

3 Problem Formulation

3.1 Problem Description

We consider a parallel dedicated machine scheduling problem, where n jobs are processed on m machines, each job belonging to a family and processed on a single machine without preemption. A set of parts which use the same die is a family and a sequence dependent setup must be performed between jobs belonging to different families. Operators are required to process jobs and setup teams are required to perform the machine setups. Operators and setup teams are renewable and dynamic resources. The number of operators for each shift is known in advance, and the system must provide scheduling solutions at the start of each shift. The objective of the Injection Moulding Department is to meet the daily demand for parts within the constraints of capacity, and the availability of operators and setup teams, minimizing the maximum completion time for each machine.

3.2 Model Formulation

In this section we formally introduce the dedicated parallel machine with setups and resources. In this paper we use small letters to refer to parameters and capital letters to refer to variables. The sets of variables and parameters used are the following:

- A set of m machines that can process the jobs, denoted as $M = \{1, \dots, m\}$, indexed by i .
- A set of $m + n$ jobs to be processed, $N = \{1, \dots, m + n\}$, indexed by j , where the first m jobs represent the initial jobs on the respective machine number.
- p_j denotes the time required to process job j .
- s_{jk} denotes the time required to do the setup between jobs j and k .
- r_{max}^P are the maximum amount of processing resources. These are required specifically for processing jobs and cannot be used for setups. We denote these resources as P-resources.
- r_{max}^S are the maximum amount of machine setup resources. These are required specifically for machine setup and cannot be used for job processing. We denote these resources as S-resources.
- r_j^P are the resources of P-resources required to do processing of job j .
- r_j^S are the resources of S-resources required to do setup of job j .

- M is a sufficiently large constant.

As the machines are dedicated, the set of N jobs is divided into m subsets $(N_1, \dots, N_m)$ so that the jobs in set N_i are only processed on machine i . Each machine has a dummy job (0) representing its start and end. Therefore, $N_{i0} = N_i \cup \{0\}$ represents the set of jobs which are to be processed on machine i plus the dummy job. Moreover, processing and setup times (p_j and s_{jk}), as well as the resources required for each operation (r_j^P and r_j^S) are deterministic. In this paper the UPMSR models based on strip-packing and indexed to time introduced by [7] are adapted to a real problem where the parallel machines are dedicated and have different release dates (the production system has an initial state), and a new objective function is proposed to minimize the sum of the machines completion times. The decision variables used are the following:

- $X_{ijk} = 1$ if job k is processed immediately after job j on machine i , zero otherwise.
- $Cmax_i$ is the completion time (minutes) of machine i .
- C_j is the completion time of job j (coordinate where processing of job j finishes in the time dimension) (minutes).
- B_j is the completion time of the setup before the processing of job j on the corresponding machine (coordinate where setup of job j finishes in the time dimension).
- $R_j^P \in [0, r_{max}^P]$ is the coordinate of processing of job j in the P-resources dimension (the specific resources employed in the processing of job j).
- $R_j^S \in [0, r_{max}^S]$ is the coordinate of setup of job j in the S-resources dimension (the specific resources employed in the setup before processing of job j).
- $D_{jk}^P, E_{jk}^P, F_{jk}^S, G_{jk}^S$ are binary auxiliary variables with value = 1 when the variable value studied in constraints (time or resources) of job k is higher than the variable value studied (time or resources) of job j . The letters in superscript are used to indicate the dimension where they are used.

The MILP strip-packing model is:

$$\text{Min } Z = \sum_{i \in M} Cmax_i \quad (1)$$

$$X_{i0i} = 1, i \in M \quad (2)$$

$$\sum_{j \in N_i, j \neq k} X_{ijk} = 1, i \in M, k \in N_i \setminus \{i\} \quad (3)$$

$$\sum_{j \in N_{i0} \setminus \{i\}, j \neq k} X_{ikj} = 1, i \in M, k \in N_i \quad (4)$$

$$C_k - C_j + M(1 - X_{ijk}) \geq s_{jk} + p_k, i \in M, j \in N_i, k \in N_i \setminus \{i\}, j \neq k \quad (5)$$

$$C_k \geq \sum_{j \in N_{i0}, j \neq k, s_{jk} > 0} X_{ijk}s_{jk} + p_k, k \in N_i \setminus \{i\}, i \in M \quad (6)$$

$$C_i \geq p_i, i \in M \quad (7)$$

$$Cmax_i \geq C_k, i \in M, k \in N_i \quad (8)$$

$$B_k - C_j + M(1 - X_{ijk}) \geq s_{jk}, i \in M, j \in N_i, k \in N_i \setminus \{i\}, j \neq k, s_{jk} > 0 \quad (9)$$

$$B_k \leq C_k - p_k, k \in N \quad (10)$$

$$r_{max}^P \geq R_k^P, k \in N \quad (11)$$

$$R_k^P \geq r_k^P, k \in N \quad (12)$$

$$r_{max}^S \geq R_k^S, k \in N \quad (13)$$

$$R_k^S \geq r_k^S, k \in N \quad (14)$$

$$C_k - C_j + M(1 - D_{jk}^P) \geq p_k, i, q \in M, q \neq i, j \in N_i, k \in N_q \quad (15)$$

$$R_k^P - R_j^P + M(1 - E_{jk}^P) \geq r_k^P, i, q \in M, q \neq i, j \in N_i, k \in N_q \quad (16)$$

$$D_{jk}^P + D_{kj}^P + E_{jk}^P + E_{kj}^P \geq 1, i, q \in M, q \neq i, j \in N_i, k \in N_q, j \neq k \quad (17)$$

$$B_k - B_j + M(1 - F_{jk}^S) \geq \sum_{l \in N_{q0}, l \neq k, s_{lk} \geq 0} X_{qlk} s_{lk}, i, q \in M, q \neq i, j \in N_i, k \in N_q \quad (18)$$

$$R_k^S - R_j^S + M(1 - G_{jk}^S) \geq r_k^S, i, q \in M, q \neq i, j \in N_i, k \in N_q \quad (19)$$

$$F_{jk}^S + F_{kj}^S + G_{jk}^S + G_{kj}^S \geq 1, i, q \in M, q \neq i, j \in N_i, k \in N_q \quad (20)$$

$$X_{ijk} \in \{0, 1\}, Y \geq 0, C_k \geq 0, B_k \geq 0, j, k \in N_i, i \in M \quad (21)$$

The objective function (1) minimizes the sum of maximum completion time for each machine. Constraints (2) ensure that the initial jobs are the first to be processed immediately after the dummy jobs on each machine. The constraints (3) and (4) make sure that only one job is processed immediately before and only one job is processed immediately after each job. Constraints (5) break subtours and give the right scheduling order, because if k is processed after j on machine i , it forces finishing the processing of job k not earlier than $s_{jk} + p_k$. Constraints (6) set the lower limit of completion time of job k (C_k) to be greater or equal than the sum of its setup plus its processing time while constraints (7) do the same for the initial jobs. Constraints (8) establish the relationship between $Cmax_i$ of machine i and the completion times of jobs on that machine. Constraints (9) force that, if job j is the previous job done before job k on machine i , the finishing time of setup k must be later than the finishing time of processing

of job j plus setup time of job k . Constraints (10) ensure that setup finishing time of job k is earlier than processing starting time of job k . Constraints (11)–(14) set the coordinate limits of specific resources used in processing (R_k^P) and used specifically in setups (R_k^S). Constraints (15) show that, if C_k is later than C_j , the difference between these completion times is greater than or equal to the processing time of job k . In (16) if R_k^P is greater than R_j^P , the difference between coordinates of these resource points is greater than or equal to the resources used in the job k processing. Constraints (17) force at least one of the binary variables to have the value of 1. Constraints (18) show that, if B_k is later than B_j , the difference between these setup finishing times is greater than or equal to the setup time of job k . In (19) if R_k^S is greater than R_j^S , the difference between coordinates of these resource points is greater than or equal to the resources used in the job k setup. Constraints (20) force at least one of the binary variables to have the value of 1. Finally, constraints (21) set the limits of the variables. In this model, in addition to several simplifications of the original model, the objective function (1) and the constraints (2) are new to reflect the specific conditions of the real problem. The MILP model indexed to time is:

- $E_{ikt} = 1$ if k is being processed on machine i in time t , zero otherwise.
- $F_{ijkt} = 1$ if k is in setup after job j on machine i in time t , zero otherwise.
- t_{max} is the planning horizon.

E_{ikt} and F_{ijkt} are decision variables and t_{max} is a parameter. For this MILP we use Eqs. (1)–(10) and (21). The other constraints are:

$$\sum_{t \leq t_{max}} E_{ikt} \geq p_k, i \in M, k \in N_i \quad (22)$$

$$C_k \geq tE_{ikt}, i \in M, k \in N_i, t \leq t_{max} \quad (23)$$

$$C_k + 1 - p_k \leq tE_{ikt} + M(1 - E_{ikt}), i \in M, k \in N_i, t \leq t_{max} \quad (24)$$

$$\sum_{t \leq t_{max}} F_{ijkt} \geq s_{jk} X_{ijk}, i \in M, j \in N_i, k \in N_i, j \neq k, s_{jk} > 0, k \geq m \quad (25)$$

$$B_k \geq \sum_{j \in N_i, j \neq k} tF_{ijkt}, i \in M, k \in N_i, t \leq t_{max}, s_{jk} > 0, k \geq m \quad (26)$$

$$B_k + 1 - \sum_{j \in N_i, j \neq k, s_{jk} > 0} s_{jk} X_{ijk} \leq \sum_{j \in N_i, j \neq k, s_{jk} > 0} tF_{ijkt} + M \left(1 - \sum_{j \in N_i, j \neq k, s_{jk} > 0} F_{ijkt} \right), i \in M, k \in N_i, t \leq t_{max} \quad (27)$$

$$R_{max}^P \geq \sum_{i \in M, k \in N_i} r_k^P E_{ikt}, t \leq t_{max} \quad (28)$$

$$R_{max}^S \geq \sum_{i \in M, j, k \in N_i, j \neq k, s_{jk} > 0} r_k^S F_{ijkt}, t \leq t_{max} \quad (29)$$

Constraints (21) and (22) work together to ensure that the E_{ikt} variable is only 1 for the exact duration of time that job k is being processed on machine i . This prevents

unnecessary idle time on the machine and ensures that each job is processed for the minimum amount of time necessary to complete it. Constraints (23) complements these constraints by requiring that when E_{ikt} is equal to 1, the current time t must be greater than or equal to the completion time of job k minus its processing time plus 1. Constraints (24)–(26) represent the same for setups, where the time considered is the setup time. Constraints (27) ensures that the sum of all resources used in processing at each instant t is less than or equal to the maximum number of resources available for processing. Constraints (28) ensures the same for setups.

4 Computational Experiments

To generate the instances, the number of jobs, number of machines, processing and setup times, number of setup teams, number of operators available, and their use by jobs were defined based on the real data from the company, and the time is in minutes. The sets of problems, small, medium, and large, are presented in Table 1. For each of the sets, 10 instances were randomly generated using processing and setup times generated from distributions of $U(20,240)$, and $U(30,50)$, respectively, and the number of operators to support the processing of jobs on the machines generated from the distribution of $U(1,4)$. Setup times must satisfy the triangular inequality. The setups require a setup team. For the time indexed model, t_{max} was set to 480 min (one shift).

Table 1. Sets of test instances

Instance	Machines	Jobs	Operators	Setup Teams
Small	4	12	8	1
Medium	8	24	16	2
Large	16	40	30	2

The CPLEX 22.1.1.0 solver was used, with programming done in Python using the Pulp library on Microsoft Visual Studio Code 2023 IDE. The computational tests were executed on an Intel Core i7-85654 CPU1.80 GHz, with 8.00 GB of RAM. To ensure reasonable runtimes, a time limit of 1800 s was set. Throughout the computational study, the indicators analysed were the gap to the optimum solution, the percentage of optimal solutions obtained, and the computation time. The performance of two models, strip-packing and time-indexed, was analysed and the average results for each set of instances are presented in Table 2. For medium and large cases, only the results for the strip-packing model are presented, as the time-index model could not find a solution within the time limit. The strip-packing model gives good results for the small and medium instances, and for the large instances it could always find a feasible solution with average gap of 14.09%. In contrast, the time-indexed model can only solve small problems.

Table 2. Comparison between strip-packing and time-indexed models

Instance	Strip-packing			Time-indexed		
	%Gap	%Opt	Time (s)	%Gap	%Opt	Time (s)
Small	0.00	100	0.91	5.6	60	1800
Medium	2.15	40	984.05	–	–	–
Large	14.09	0	1800	–	–	–

5 Conclusions and Future Research

This paper studies the special case of PMSR. To solve this problem, two previously proposed models have been adapted and a novel objective function, the minimisation of the sum of the machine completion times, is proposed to reflect the real conditions of the manufacturing environment that motivates this work. Overall, the strip-packing model outperforms the time-indexed model in solving this type of PMSR problem and shows a good potential for solving real-world problems and for future research in this area. The solutions obtained show that the new objective function provides a compact production schedule that allows the simultaneous minimisation of machine idle times and setup times for the PMSR with identical parallel machines with different release dates. To further enhance the performance of the strip-packing model, complementary approaches such as matheuristics can be explored. Matheuristics combine the strengths of mathematical programming and metaheuristics to provide a hybrid optimization approach that can efficiently solve complex problems.

References

1. Blazewicz, J., Lenstra, J.K., Kan, A.R.: Scheduling subject to resource constraints: classification and complexity. *Discret. Appl. Math.* **5**, 11–24 (1983)
2. Błażewicz, J., Kubiak, W., Röck, H., Szwarcfiter, J.: Minimizing mean flow-time with parallel processors and resource constraints. *Acta Inform.* **24**(5), 513–524 (1987). <https://doi.org/10.1007/BF00263292>
3. Ventura, J.A., Kim, D.: Parallel machine scheduling about an unrestricted due date and additional resource constraints. *IIE Trans.* **32**, 147–153 (2000)
4. Akyol Ozer, E., Sarac, T.: MIP models and a matheuristic algorithm for an identical parallel machine scheduling problem under multiple copies of shared resources constraints. *TOP* **27**, 94–124 (2019)
5. Bektur, G., Saraç, T.: A mathematical model and heuristic algorithms for an unrelated parallel machine scheduling problem with sequence-dependent setup times, machine eligibility restrictions and a common server. *Comput. Oper. Res.* **103**, 46–63 (2019)
6. Pinheiro, J.C., Arroyo, J.E.C., Fialho, L.B.: Scheduling unrelated parallel machines with family setups and resource constraints to minimize total tardiness. In: *Proceedings of the 2020 Genetic and Evolutionary Computation Conference Companion*, pp. 1409–1417 (2020)
7. Fanjul-Peyro, L.: Models and an exact method for the unrelated parallel machine scheduling problem with setups and resources. *Expert Syst. Appl.* **5**, 100022 (2020)

8. Yepes-Borrero, J.C., Villa, F., Perea, F., Caballero-Villalobos, J.P.: GRASP algorithm for the unrelated parallel machine scheduling problem with setup times and additional resources. *Expert Syst. Appl.* **141**, 112959 (2020)
9. Yepes-Borrero, J.C., Perea, F., Ruiz, R., Villa, F.: Bi-objective parallel machine scheduling with additional resources during setups. *Eur. J. Oper. Res.* **292**, 443–455 (2021)
10. Zhang, L., Deng, Q., Lin, R., Gong, G., Han, W.: A combinatorial evolutionary algorithm for unrelated parallel machine scheduling problem with sequence and machine-dependent setup times, limited worker resources and learning effect. *Expert Syst. Appl.* **175**, 114843 (2021)
11. Lopez-Esteve, A., Perea, F., Yepes-Borrero, J.C.: GRASP algorithms for the unrelated parallel machines scheduling problem with additional resources during processing and setups. *Int. J. Prod. Res.* 1–17 (2022)
12. Yunusoglu, P., Topaloglu Yildiz, S.: Constraint programming approach for multi-resource-constrained unrelated parallel machine scheduling problem with sequence-dependent setup times. *Int. J. Prod. Res.* **60**, 2212–2229 (2022)
13. Afzalirad, M., Rezaeian, J.: Resource-constrained unrelated parallel machine scheduling problem with sequence dependent setup times, precedence constraints and machine eligibility restrictions. *Comput. Ind. Eng.* **98**, 40–52 (2016)